Estudo Claro

Plataforma de estudo com IA, revisão ativa e acessibilidade

Documentação do Desenvolvedor

Arquitetura, banco de dados, API, algoritmos, IA, front-end, segurança, testes e extensão

DOCUMENTAÇÃO TÉCNICA COMPLETA

Versão 1.0 do documento · 17 de outubro de 2026
Documentação do sistema Estudo Claro (PHP 8.1+ · SQLite · PWA)

Sumário

1. Visão geral e arquitetura 4

1.1 Tecnologias 4

1.2 Princípios de projeto 4

1.3 Diagrama de componentes 4

1.4 Decisões de arquitetura e seus custos 5

2. Estrutura do projeto 6

2.1 Árvore de arquivos 6

2.2 Arquivos e responsabilidades 7

3. Ciclo de uma requisição 9

3.1 Páginas 9

3.2 API 9

3.3 Códigos de resposta 9

3.4 Execução em segundo plano 10

4. Banco de dados 11

4.1 Diagrama lógico 11

4.2 Tabelas e colunas 12

4.3 Migração 12

4.4 Chaves da tabela settings 13

4.5 Colunas JSON 13

5. Referência da API 14

6. Algoritmos e regras de negócio 19

6.1 Repetição espaçada (SM-2 adaptado) 19

6.2 XP, níveis, estrelas e desbloqueio 19

6.3 Minutos estudados, ritmo e estimativa de conclusão 19

6.4 Jogo bônus: validação no servidor 20

6.5 Limites e proteções por contagem 20

7. Camada de IA 21

7.1 Provedores e adaptadores 21

7.2 Contingência e disjuntor 21

7.3 Extração e normalização de respostas 22

7.4 Orquestrador de trilhas (run_route_job) 22

7.5 Modo agente e esqueleto 23

7.6 Prompts: contrato de saída . . . 23

8. Aulas em áudio e vídeo . . . 24

8.1 Upload em partes . . . 24
8.2 Pipeline (run_media_job) . . . 24
8.3 Transcrição (STT) . . . 25

9. Mapa de conhecimento e aprofundamento . . . 26

9.1 Dados do grafo (graph_data) . . . 26
9.2 Renderização (front-end) . . . 26
9.3 Aprofundamento por tópico . . . 26

10. Front-end . . . 28

10.1 Organização do app.js . . . 28
10.2 Roteador: rotas e telas . . . 29
10.3 Edições, temas e desenho padrão . . . 29
10.4 Telas principais . . . 29
10.5 PWA e cache . . . 30
10.6 Segurança no navegador . . . 30

11. Instalador . . . 31

12. Segurança técnica . . . 32

13. Formatos de dados . . . 33

13.1 Pacote de conteúdo (importação pelo administrador) . . . 33
13.2 Trilha exportada (.trilha.json) . . . 33
13.3 Resposta de uma aula (media) . . . 33

14. Testes e garantia de qualidade . . . 34

14.1 Estratégia . . . 34
14.2 Como a prévia é montada . . . 34
14.3 Verificações estáticas feitas no PHP . . . 34
14.4 O que falta testar (recomendado antes da produção) . . . 35

15. Como estender o sistema . . . 36

15.1 Nova rota da API . . . 36
15.2 Nova tela . . . 36
15.3 Novo provedor de IA . . . 36
15.4 Novo jogo bônus . . . 36
15.5 Nova conquista, tema ou nível . . . 36

15.6 Nova edição (versão da interface) . . . 37

16. Operação, desempenho e evolução . . . 38

16.1 Configuração de produção . . . 38

16.2 Desempenho . . . 38

16.3 Escalabilidade e migração de banco . . . 38

16.4 Observabilidade . . . 38

16.5 Limitações conhecidas . . . 39

Apêndice A — Glossário técnico . . . 40

Apêndice B — Lista de verificação para revisão de código . . . 41

1. Visão geral e arquitetura

O Estudo Claro é uma aplicação web de estudo com repetição espaçada, trilhas geradas por IA, gamificação e acessibilidade. É propositalmente simples de hospedar: **PHP 8.1+ sem framework, SQLite e um front-end em JavaScript puro**, sem etapa de compilação, sem dependências externas e sem CDN. Este documento descreve a estrutura, as regras, os formatos e os pontos de extensão com o detalhe necessário para manter e evoluir o sistema.

1.1 Tecnologias

Camada	Escolha	Observações
Linguagem do servidor	PHP 8.1+ (strict_types)	Tipo de retorno never, operador ??=, funções de seta. Sem Composer.
Banco	SQLite via PDO, modo WAL	Um arquivo; chaves estrangeiras ligadas; busy_timeout de 5 s.
API	JSON sobre HTTP, uma só entrada (api.php?r=rota)	GET para leitura, POST para escrita com cabeçalho X-CSRF.
Front-end	SPA em JavaScript puro (ES2020), roteador por hash	Todo texto entra por textContent; sem innerHTML com dados.
Estilo	CSS com variáveis; três camadas (base, portal, padrão)	Sem estilos inline por causa da CSP (style-src self).
PWA	Service worker + manifesto	Casca em cache; API sempre na rede; fila offline de notas.
IA	cURL para Anthropic, Gemini e compatíveis com OpenAI	Contingência entre provedores, disjuntor e modo agente.
Gráficos	SVG (mapa da trilha) e Canvas 2D (grafo 3D e jogos)	Sem bibliotecas.
Testes	Playwright (Chromium) contra uma prévia com servidor simulado	Veja o capítulo 14.

1.2 Princípios de projeto

- **Acessibilidade primeiro**: HTML semântico, foco gerenciado, leitura em voz alta, alto contraste, listas equivalentes para telas gráficas e respeito a prefers-reduced-motion.
- **Segredos só no servidor**: chaves de IA nunca chegam ao navegador; a API devolve apenas os 4 últimos caracteres.
- **Degradação graciosa**: se a IA cai, há modo agente e esqueleto; se a transcrição cai, há modo manual; se não há internet, a revisão continua.
- **Sem build e sem CDN**: o que está na pasta é o que roda; CSP estrita (default-src self).
- **Servidor como autoridade**: XP do jogo, limites diários, estrelas e permissões são calculados ou verificados no servidor.
- **Um arquivo de banco**: instalação, cópia e restauração triviais; o esquema se autoatualiza (ensure_schema).

1.3 Diagrama de componentes

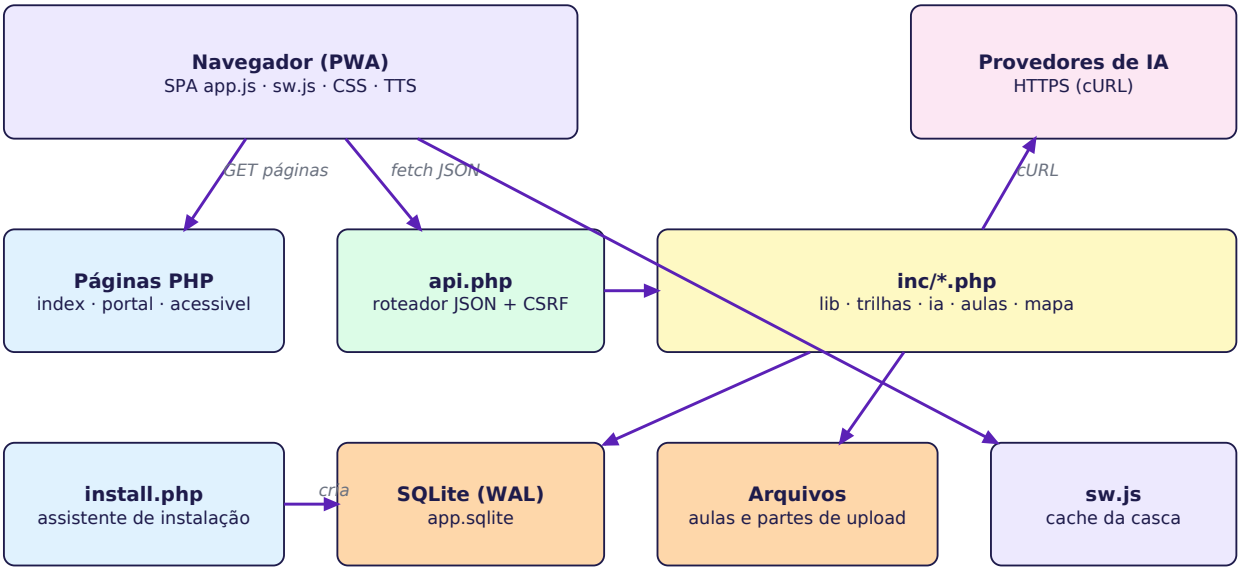


Figura 1 — Componentes e fluxos. O navegador conversa só com as páginas PHP e com api.php; as chamadas a provedores de IA saem do servidor.

1.4 Decisões de arquitetura e seus custos

Decisão	Por quê	Custo / limite
SQLite em vez de MySQL	Hospedagem compartilhada simples, backup por cópia de arquivo, zero configuração.	Um único escritor por vez; escala para centenas de usuários ativos, não para milhares simultâneos (veja 16.3).
Sem framework PHP	Poucas dependências, leitura direta do código, baixo risco de quebrar em atualizações de hospedagem.	Roteamento e validação são manuais (switch em api.php).
SPA com roteador por hash	Funciona em hospedagem estática, sem regras de reescrita; offline simples.	Um arquivo grande (app.js); sem divisão em módulos.
Trabalho em segundo plano na própria requisição	Sem fila nem cron: bg_start responde e continua o trabalho.	Depende de fastcgi_finish_request e do tempo máximo do PHP; por isso há rotas de retomada (route_retry, media_retry).
Chaves de IA no banco	Administrador troca pelo painel, sem editar arquivos.	Quem lê o banco lê as chaves: proteger a pasta de dados e as cópias de segurança.
CSP sem inline	Reduz muito o impacto de XSS.	O JS aplica estilos dinâmicos pelo CSSOM (element.style), nunca por atributo style em HTML.

2. Estrutura do projeto

2.1 Árvore de arquivos

```
estudo-claro/
├── index.php  portal.php  acessivel.php   páginas (casca HTML)
├── install.php                            assistente de instalação
├── api.php                                API JSON
├── sw.js  manifest.webmanifest  .htaccess  LEIA-ME.txt
├── pacotes/direito-administrativo.json    pacote de exemplo
├── assets/
│   ├── app.js  app.css  portal.css  std.css
│   ├── install.js  install.css  theme.js
│   └── mark.svg  logo.svg  icon.svg
└── inc/
    ├── lib.php  shell.php  trilhas.php
    ├── ia.php  aulas.php  mapa.php
    ├── preconfig.php   (apagado na instalação)
    └── .htaccess

(fora da pasta pública, criada pelo instalador)
estudo-claro_dados/
├── app.sqlite  app.sqlite-wal  app.sqlite-shm
└── media/<id>/   partes do upload, arquivo da aula, áudio convertido
```

2.2 Arquivos e responsabilidades

Arquivo	Linhas	Responsabilidade
`.htaccess`	11	Regras do Apache: sem listagem, nega config.php, install.key e SQLite.
`LEIA-ME.txt`	48	Instruções resumidas de instalação.
`acessivel.php`	5	Atalho que fixa a edição Mobile acessível e chama a casca.
`api.php`	489	Roteador da API JSON (switch por ?r=): sessão, CSRF, autenticação, todas as rotas, execução em segundo plano.
`index.php`	15	Porta de entrada: se instalado, carrega a casca e escolhe a edição (portal ou acessível); senão envia ao instalador.
`install.php`	135	Assistente de instalação em 5 passos; endpoint de teste de agentes (?do=test); grava config.php.
`manifest.webmanifest`	1	Manifesto PWA (nome, ícones, cores, modo standalone).
`portal.php`	5	Atalho que fixa a edição Portal e chama a casca.
`sw.js`	10	Service worker: cache da casca do app, navegação offline, nunca guarda a API.
`assets/app.css`	246	CSS base acessível: variáveis, temas, componentes.
`assets/app.js`	1109	SPA completa em JavaScript puro: roteador, telas, gamificação, mapa da trilha, grafo 3D, jogos, TTS, fila offline.
`assets/icon.svg`	5	Ícone do aplicativo.
`assets/install.css`	27	Estilos do instalador.
`assets/install.js`	68	Lógica do assistente (agentes, teste, cadeia de socorro).
`assets/logo.svg`	7	Logomarca completa.
`assets/mark.svg`	4	Marca (ícone) vetorizada.
`assets/portal.css`	84	Camada visual do Portal (cores vivas, menu lateral, animações).
`assets/std.css`	73	Camada do “desenho padrão” (acessibilidade desligada).
`assets/theme.js`	2	Aplica tema, tamanho de letra e classes no <html> antes do JS principal (evita piscar).
`inc/.htaccess`	2	Nega acesso direto às bibliotecas.
`inc/aulas.php`	202	Aulas: upload em partes, STT (Gemini e Whisper), ffmpeg, estruturação, bibliografia, aprofundamento.
`inc/ia.php`	291	Provedores de IA, contingência, disjuntor, normalização, prompts, orquestrador de trilhas.
`inc/lib.php`	133	Núcleo: configuração, banco, sessão, CSRF, preferências, SM-2, esquema, instalação de pacotes.
`inc/mapa.php`	70	Dados do grafo, configuração do jogo e do mapa, aprofundamento por tópico.
`inc/preconfig.php`	19	Credenciais pré-configuradas (apagado ao concluir a instalação).
`inc/shell.php`	41	Casca HTML das duas edições; cabeçalhos de segurança (CSP); escolha da edição.
`inc/trilhas.php`	101	Settings, planos, visibilidade, exportar/importar, clonar, dados do painel da trilha.
`pacotes/direito-administrativo.json`	575	Pacote de exemplo com bibliografia detalhada.

Arquivo	Linhas	Responsabilidade
`Total`	**3778**	

> **Nota** As linhas do código são longas porque o projeto usa um estilo compacto (várias instruções por linha em trechos de SQL e de normalização). Use um formatador se for ampliar muito o código.

3. Ciclo de uma requisição

3.1 Páginas

`index.php`, `portal.php` e `acessivel.php` carregam `inc/shell.php`, que: (1) redireciona ao instalador se não há config.php; (2) executa ensure_schema(); (3) lê a configuração `editions` (ambas, portal, acessivel) e decide a edição, também por cookie e parâmetro ?trocar=1; (4) emite os cabeçalhos de segurança e a casca HTML com a classe de edição no body; (5) carrega theme.js e app.js. A tela de escolha é desenhada pelo próprio app.js (vChooser) quando não há edição definida.

```
Content-Security-Policy: default-src 'self'; img-src 'self' data:; style-src 'self';
  script-src 'self'; connect-src 'self'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'
```

3.2 API

Toda chamada vai a `api.php?r=<rota>`. O fluxo, na ordem:

1. Carrega lib, trilhas, ia, aulas e mapa; define Content-Type JSON, Cache-Control no-store e nosniff.
2. Se não instalado, responde 503.
3. boot_session() (cookie ec: httponly, SameSite Lax, Secure em HTTPS, path da aplicação) e ensure_schema() (uma vez por requisição).
4. Lê a rota ($_GET[r], padrão boot). Em POST, decodifica o corpo JSON (exceto media_chunk, que envia bytes brutos).
5. Em POST, exige o cabeçalho X-CSRF igual ao token da sessão, senão 419.
6. O switch executa a rota: user() devolve 401 sem sessão, admin() devolve 403 sem perfil; os dados saem por out($dados, $codigo), que termina a execução (tipo never).

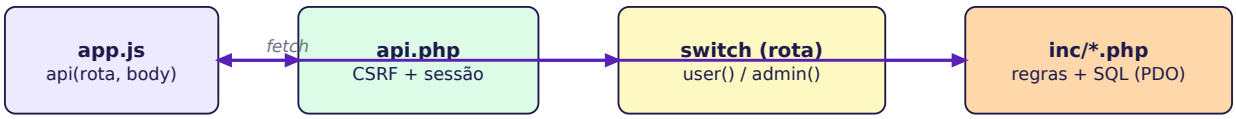


Figura 2 — Fluxo da API. A resposta de volta é sempre JSON; erros trazem {error: mensagem} com o código HTTP.

3.3 Códigos de resposta

Código	Quando
200	Sucesso.
400	Entrada inválida (mensagem em error).
401	Sem sessão: o front limpa o usuário e mostra o login.
403	Sem permissão (não é admin, não é o autor, senha atual errada).
404	Recurso inexistente ou invisível para o usuário.
419	Token CSRF ausente ou inválido (sessão expirada).
423	Recurso bloqueado (por exemplo, trilha desativada ao abrir a fila de revisão).
429	Excesso de tentativas, limite de trilhas por IA ou de aulas simultâneas.
503	Sistema ainda não instalado.

3.4 Execução em segundo plano

```
function bg_start(array $payload, callable $fn): never {
    ignore_user_abort(true); set_time_limit(0); session_write_close();
    $body = json_encode($payload); while (ob_get_level()) ob_end_clean();
    header('Content-Length: ' . strlen($body)); header('Connection: close'); echo $body;
    if (function_exists('fastcgi_finish_request')) fastcgi_finish_request(); else flush();
    $fn(); exit;     // continua o trabalho depois de responder
}
```

Usado por route_create (orquestrador de trilhas) e media_done (pipeline de aulas): o navegador recebe o id do trabalho na hora e consulta o andamento com `route_job` ou `media_job` a cada poucos segundos. O progresso fica no banco (colunas progress), então sobrevive à troca de página. Se o processo for morto pela hospedagem, o trabalho fica parado e o usuário usa “Tentar de novo” (rotas route_retry e media_retry), que retomam do ponto salvo.

4. Banco de dados

4.1 Diagrama lógico

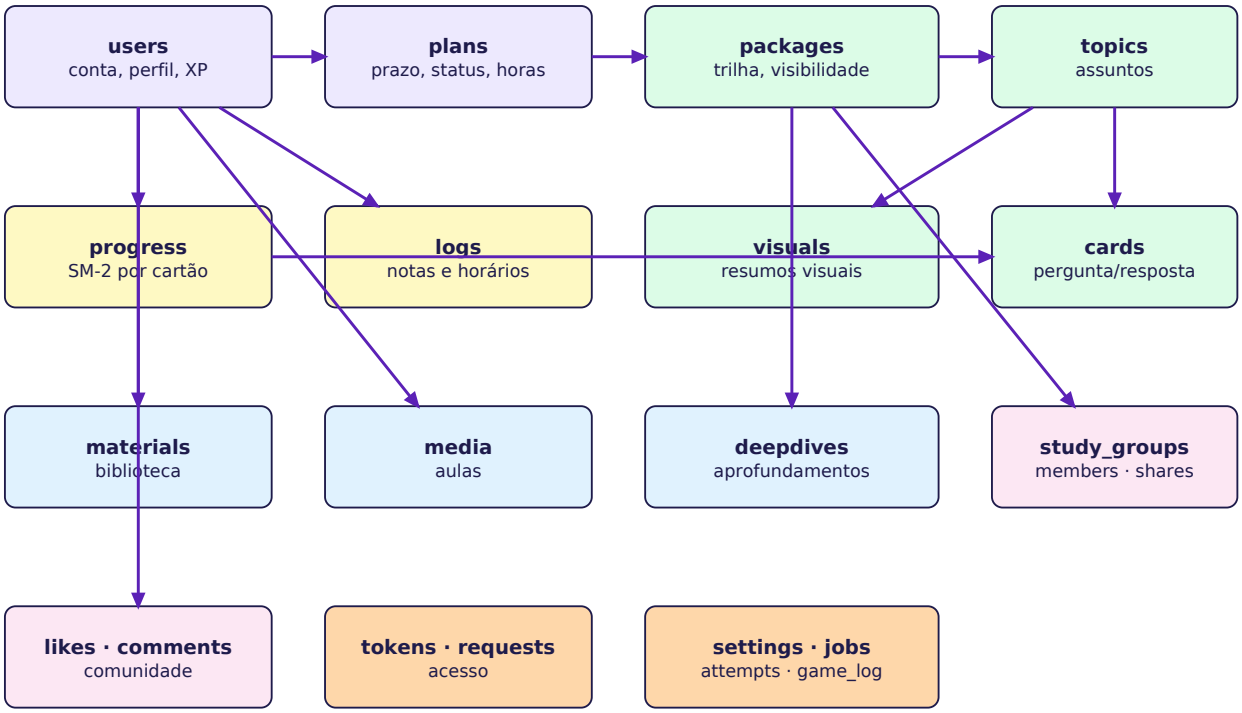


Figura 3 — Relações principais. Todas as chaves estrangeiras usam ON DELETE CASCADE, então apagar um usuário ou uma trilha leva junto o que depende dele.

4.2 Tabelas e colunas

Tabela	Colunas (tipo) — notas
`users`	id, name, email (único), pass (hash), role (aluno|admin), status (ativo|bloqueado), prefs (JSON), created, xp, pkg (trilha atual)
`packages`	id, slug (único), name, goal, emoji, created, owner_id (NULL = do sistema), kind (sistema|ia|importada|…), objective, strategy (JSON), biblio (JSON), sources (JSON), visibility (privada|grupos|publica), tags (JSON), origin_id
`topics`	id, package_id, title, law, emoji, summary (blocos separados por quebra de linha), mnemonic, pos, difficulty (1–5), refs (JSON)
`visuals`	id, topic_id, emoji, title, points (JSON), pitfall, example, pos
`cards`	id, topic_id, q, a
`progress`	id, user_id, card_id, ease (REAL), ivl (dias), reps, lapses, due (epoch); único (user_id, card_id)
`logs`	id, user_id, card_id, grade (0–3), ts (epoch)
`plans`	id, user_id, package_id, deadline, hpw, start_ts, status (ativa|pausada|excluida), prev_status, paused_at, paused_total, deleted_at
`tokens`	id, token (único), role, used_by, expires, created
`requests`	id, name, email, msg, status (pendente|aprovado|recusado), token, created
`jobs`	id, user_id, status, msg, route_id, created, payload (JSON), progress (JSON)
`settings`	k (chave), v (texto)
`attempts`	id, k, ts — janelas de limite de tentativas
`study_groups`	id, name, descr, owner_id, code (único), created
`group_members`	group_id, user_id, role (dono|membro), joined
`route_shares`	package_id, group_id
`likes`	package_id, user_id
`comments`	id, package_id, user_id, text, created
`materials`	id, user_id, title, author, kind (texto|link|video|pdf|aula), url, text, tags, pdf
`media`	id, user_id, title, filename, mime, size, stored, status (enviando|processando|manual|pronto|erro), msg, progress (JSON), opts (JSON), transcript, structure (JSON), biblio (JSON), deep (JSON), material_ids (JSON)
`deepdives`	id, user_id, package_id, topic_id, card_id, kind (card|tema), title, q, a, summary, points (JSON), refs (JSON), material_id
`game_log`	id, user_id, game, score, xp, ts

Índices: `i_deep(deepdives: user_id, package_id)`, `i_logs(logs: user_id, ts)`, `i_prog(progress: user_id, due)`, `i_att(attempts: k, ts)`. PRAGMAs na abertura: foreign_keys=ON, journal_mode=WAL, busy_timeout=5000, synchronous=NORMAL.

4.3 Migração

`ensure_schema()` roda uma vez por requisição: executa `schema()` (todos os CREATE TABLE/INDEX IF NOT EXISTS) e depois, para cada tabela com colunas acrescentadas ao longo do tempo, compara com `PRAGMA table_info` e aplica `ALTER TABLE … ADD COLUMN` no que faltar. As colunas adicionadas por migração são: packages (owner_id, kind, objective, strategy, biblio, sources, visibility, tags, origin_id), jobs (payload, progress) e topics (difficulty, refs). Para criar nova coluna, acrescente-a ao array $add de lib.php; nunca altere o tipo de uma existente (SQLite não permite sem recriar a tabela).

4.4 Chaves da tabela settings

Chave	Padrão	Conteúdo
`editions`	ambas	ambas | portal | acessivel.
`game_cfg`	(padrões de game_cfg())	JSON do jogo bônus, saneado por clean_game().
`graph_cfg`	(padrões de graph_cfg())	JSON do mapa, saneado por clean_graph().
`media_max_mb`	500	Tamanho máximo de aula (10 a 4000).
`ia_enabled`	1	1 liga a geração automática; 0 força modo agente.
`ia_limit`	3	Trilhas por IA por usuário em 24 h (1 a 50).
`ia_providers`	(vazio)	JSON com até 8 provedores (id, type, label, key, model, base_url, role, search, stt, enabled).
`ia_health`	{}	Disjuntor: {id: {fails, until}}. Zerado ao salvar a configuração.
`ia_key, ia_model`	(legado)	Formato antigo de um único provedor; migrado em tempo de leitura por ia_providers().

4.5 Colunas JSON

Coluna	Formato
`users.prefs`	{theme: claro|noite|azul, fs: 16–48, lh, ls, tts, rate, a11y}; saneado por clean_prefs().
`packages.strategy`	[{tecnica, como, frequencia}]
`packages.biblio`	[{tipo, titulo, autores, ano, url, por_que, verificada}]
`packages.sources`	{origem, rascunho, copiada_de: {id, trilha, autor}, …} — metadados de procedência.
`topics.refs, packages.tags`	Lista de textos (títulos de obras / etiquetas, no máximo 8 etiquetas).
`visuals.points`	Lista de textos curtos.
`jobs.progress / media.progress`	{steps: [{k, t, s: espera|andamento|ok|erro, d}], agentes: {etapa: provedor}, avisos: []}

5. Referência da API

Esta tabela é gerada a partir do próprio api.php: a coluna **Entrada** lista os campos lidos de $in (corpo JSON) e de $_GET; **Acesso** indica se a rota chama user() ou admin(). Em POST, o cabeçalho X-CSRF é obrigatório. Respostas de erro: {error}.

Sessão e conta

Rota	M	Acesso	Entrada	O que faz
`boot`	G	público	–	Estado inicial: usuário (ou nulo), CSRF, nome, edições, configuração do jogo e do mapa.
`login`	P	público	`email, pass`	Entra com e-mail e senha; limite de 8 tentativas em 10 min; renova a sessão.
`register`	P	público	`token, email, name, pass`	Cria a conta consumindo um token válido (senha de 8+ caracteres).
`request`	P	público	`email, name, msg`	Pedido de acesso (nome, e-mail, mensagem) para o administrador.
`pwd_change`	P	usuário	`current, new`	Troca a própria senha (confere a atual; nova diferente; 6 tentativas em 10 min).
`logout`	P	público	–	Encerra a sessão.
`prefs`	P	usuário	–	Salva preferências de leitura e acessibilidade (clean_prefs).
`stats`	G	usuário	–	XP, nível, sequência de dias, estrelas por assunto e conquistas.
`pkg_set`	P	usuário	`id`	Define a trilha atual do usuário.

Estudo

Rota	M	Acesso	Entrada	O que faz
`topics`	G	usuário	–	Assuntos da trilha atual com estrelas e contagens.
`topic`	G	usuário	`id`	Um assunto: resumo, mnemônico, referências e cartões.
`visuals`	G	usuário	`id`	Resumos visuais de um assunto.
`queue`	G	usuário	`mode, id`	Fila de revisão (modos due, topic, exam) com cartões vencidos e novos; 423 se a trilha está desativada.
`grades`	P	usuário	`grades`	Recebe um lote de notas (cartão, nota 0–3), aplica o SM-2 e devolve o novo estado; usado também pela fila offline.

Trilhas

Rota	M	Acesso	Entrada	O que faz
`routes`	G	usuário	–	Trilhas do usuário (sistema, criadas, importadas) com plano e andamento.
`route_data`	G	usuário	`id`	Dados do painel da trilha: assuntos, plano, minutos estudados, semana, compartilhamentos.
`route_plan`	P	usuário	`id, deadline, hpw`	Altera prazo e horas por semana da trilha.
`route_state`	P	usuário	`id, act`	Desativa, reativa ou exclui a trilha (excluir envia à lixeira do administrador).
`route_export`	G	usuário	`id`	Gera o arquivo .trilha.json com integridade SHA-256.
`route_import`	P	usuário	`json`	Importa .trilha.json ou pacote simples; restaura progresso e plano; avisa se o arquivo foi editado.
`route_create`	P	usuário	`deadline, materials, material_ids, videos, mode, theme, objective, level, themes…`	Valida os dados, aplica o limite diário e inicia o orquestrador em segundo plano; devolve o id do trabalho.
`route_job`	G	usuário	`id`	Andamento de um trabalho de criação (etapas, agentes, resultado).
`route_manual`	P	usuário	`job, text`	Modo agente: recebe o JSON colado, normaliza e cria a trilha.
`route_skeleton`	P	usuário	`job, text`	Cria um esqueleto sem IA a partir dos temas.
`route_retry`	P	usuário	`job, text`	Retoma um trabalho de criação interrompido.
`ia_status`	G	usuário	–	Se a IA automática está disponível, limite e uso do usuário, prompt para o modo agente.
`topic_expand`	P	usuário	`id, focus, text`	Gera mais cartões e resumo visual para um assunto (IA ou prompt para o modo agente).
`topic_expand_apply`	P	usuário	`id, focus, text`	Aplica o resultado do modo agente na expansão de um assunto.

Mapa, aprofundamento e jogo

Rota	M	Acesso	Entrada	O que faz
`graph_data`	G	usuário	`id`	Nós e ligações do mapa de conhecimento (tópicos, cartões, bibliografia, aprofundamentos, trilhas relacionadas).
`deep_generate`	P	usuário	`kind, focus, topic_id, card_id, text`	Gera aprofundamento (card ou tema) de um tópico ou cartão; sem IA devolve o prompt.
`deep_apply`	P	usuário	`kind, focus, topic_id, card_id, text`	Aplica a resposta colada (modo agente) e grava o aprofundamento.
`deep_list`	G	usuário	`id`	Lista a biblioteca de aprofundamento da trilha.
`deep_del`	P	usuário	`id, with_material`	Apaga um aprofundamento.
`game_result`	P	usuário	`score, game`	Recebe a pontuação do jogo; o servidor valida a sequência de acertos e os limites e credita o XP.

Comunidade e grupos

Rota	M	Acesso	Entrada	O que faz
`explore`	G	usuário	`q, kind, limit, sort`	Busca trilhas públicas (texto, tipo, ordem popularidade/data).
`route_public`	G	usuário	`id`	Página de uma trilha da comunidade com curtidas, comentários e seguidores.
`route_like`	P	usuário	`id`	Curte ou descurte.
`route_comment`	P	usuário	`id, text`	Comenta numa trilha visível.
`comment_del`	P	usuário	`id`	Apaga comentário (admin, autor ou dono da trilha).
`route_follow`	P	usuário	`id`	Passa a acompanhar o autor.
`route_unfollow`	P	usuário	`id`	Deixa de acompanhar.
`route_clone`	P	usuário	`id`	Copia uma trilha pública ou compartilhada para a conta do usuário.
`route_visibility`	P	usuário	`id, visibility, tags, groups`	Muda visibilidade e etiquetas; só o autor (o admin pode despublicar).
`groups`	G	usuário	–	Grupos do usuário.
`group_create`	P	usuário	`name, descr`	Cria grupo e gera o código de convite.
`group_join`	P	usuário	`code`	Entra por código.
`group_data`	G	usuário	`id`	Membros e trilhas compartilhadas.
`group_leave`	P	usuário	`id`	Sai do grupo.
`group_del`	P	usuário	`id`	Exclui o grupo (dono ou admin).

Materiais e aulas

Rota	M	Acesso	Entrada	O que faz
`materials`	G	usuário	–	Lista a biblioteca de materiais.
`material_get`	G	usuário	`id`	Um material completo.
`material_save`	P	usuário	`id, title, kind, tags, pdf, author, url, text`	Cria ou edita material (texto, link, vídeo, PDF até 4 MB).
`material_del`	P	usuário	`id`	Exclui material.
`media_init`	P	usuário	`size, name, title, mime, lang, deep, biblio`	Valida extensão, tamanho e limite de 3 aulas simultâneas; cria o registro e devolve o id.
`media_chunk`	P*	usuário	`id, n`	Recebe uma parte de 4 MB (corpo bruto, id e n na URL).
`media_done`	P	usuário	`id`	Confere o arquivo montado e inicia o pipeline em segundo plano.
`media_job`	G	usuário	`id`	Andamento da aula (etapas, agentes, status).
`media_list`	G	usuário	–	Aulas do usuário.
`media_get`	G	usuário	`id`	Aula pronta: transcrição, estrutura, bibliografia, aprofundamento.
`media_text`	P	usuário	`id, text`	Modo manual: recebe a transcrição colada ou .srt/.vtt e continua.
`media_retry`	P	usuário	`id`	Retoma o pipeline.
`media_del`	P	usuário	`id, with_materials`	Exclui a aula e seus arquivos.

Administração

Rota	M	Acesso	Entrada	O que faz
`a_users`	G	admin	–	Lista usuários.
`a_user`	P	admin	`id, act, pass`	Bloqueia/desbloqueia, redefine senha ou exclui (não a própria conta).
`a_tokens`	G	admin	–	Lista tokens.
`a_token`	P	admin	`act, n, days, role, id`	Gera 1–50 tokens (validade, perfil) ou exclui não usados.
`a_reqs`	G	admin	–	Lista pedidos de acesso.
`a_req`	P	admin	`id, act`	Aprova (gera token de 7 dias), recusa ou exclui.
`a_content`	G	admin	–	Assuntos e cartões da trilha atual do admin.
`a_topic`	P	admin	`id, act, title, law, summary, mnemonic`	Cria, edita ou apaga assunto.
`a_card`	P	admin	`id, act, q, a, topic_id`	Cria, edita ou apaga cartão.
`a_pkgs`	G	admin	–	Lista pacotes.
`a_pkg_import`	P	admin	`json`	Instala pacote a partir de JSON (até 3 MB).
`a_pkg_del`	P	admin	`id`	Remove pacote (mantém ao menos um).
`a_cfg`	G	admin	–	Lê as edições oferecidas.
`a_cfg_set`	P	admin	`editions`	Define as edições (ambas, portal, acessivel).
`a_game`	G	admin	–	Lê configuração do jogo e do mapa.
`a_game_set`	P	admin	`game, graph`	Salva configuração do jogo e do mapa (saneada).
`a_ia`	G	admin	–	Provedores (chave só com 4 últimos caracteres), limites, saúde, ffmpeg.
`a_ia_save`	P	admin	`providers, media_max_mb, enabled, limit`	Salva até 8 provedores; chave em branco preserva a anterior; zera a saúde.
`a_ia_test`	P	admin	`id`	Teste de um provedor com uma resposta curta.
`a_dels`	G	admin	–	Trilhas na lixeira.
`a_del`	P	admin	`id, act`	Restaura (restore) ou apaga de vez (approve).

Nota M: G = GET, P = POST (JSON). media_chunk é POST com corpo binário (parte do arquivo) e id e n na URL. Todas as rotas aceitam apenas os campos listados; campos extras são ignorados.

6. Algoritmos e regras de negócio

6.1 Repetição espaçada (SM-2 adaptado)

Cada par (usuário, cartão) guarda **ease** (facilidade, inicia em 2,5), **ivl** (intervalo em dias), **reps** (acertos seguidos), **lapses** (erros) e **due** (próxima data, em epoch). As notas são 0 Errei, 1 Difícil, 2 Bom e 3 Fácil. A função `review()` aplica a regra, grava em progress e logs e credita o XP.

```
function review(int $uid, int $cid, int $g, int $ts): int {
    $p = (linha de progress) ?: ['ease'=>2.5,'ivl'=>0,'reps'=>0,'lapses'=>0];
    if ($g === 0) { $r = 0; $l++; $e = max(1.3, $e - 0.2); $iv = 0; }
    else {
        if     ($g === 1) { $iv = max(1, round(max(1,$iv) * 1.2)); $e = max(1.3, $e - 0.15); }
        elseif ($g === 2) { $iv = $r === 0 ? 1 : ($r === 1 ? 3 : round($iv * $e)); }
        else              { $iv = $r === 0 ? 3 : round(max(3,$iv) * $e * 1.3); $e += 0.15; }
        $r++;
    }
    $due = $g === 0 ? $ts + 600 : $ts + $iv * 86400;     // erro: volta em 10 minutos
    // UPSERT em progress; INSERT em logs; users.xp += [2,6,10,12][$g]
    return $iv;
}
```

Nota	Intervalo novo	Facilidade	Observação
0 Errei	0 (volta em 10 min)	−0,20 (mín. 1,3)	reps volta a 0; lapses +1.
1 Difícil	máx(1, ivl) × 1,2	−0,15 (mín. 1,3)	reps +1.
2 Bom	1 dia (reps 0), 3 dias (reps 1), depois ivl × ease	inalterada	reps +1.
3 Fácil	3 dias (reps 0), depois máx(3, ivl) × ease × 1,3	+0,15	reps +1.

Um cartão é **dominado** quando reps ≥ 2. A fila (queue) tem três modos: *due* (cartões vencidos ou ainda sem progresso da trilha atual), *topic* (até 8 cartões sorteados de um assunto) e *exam* (simulado final: 10 cartões sorteados da trilha; o front só oferece após 2 estrelas em todas as fases). Se a trilha está desativada, responde 423.

6.2 XP, níveis, estrelas e desbloqueio

```
XP por nota: [2, 6, 10, 12]
nível      = floor( sqrt(xp / 30) ) + 1            // limiar do nível L: 30 × (L-1)²
estrelas   = pc >= 1 ? 3 : pc >= .66 ? 2 : pc >= .33 ? 1 : 0   // pc = dominados / total do assunto
fase i abre se i == 0 ou estrelas(i-1) >= 1
castelo (simulado final) abre se todas as fases têm >= 2 estrelas; capelo se todas têm 3
reforço (weak) = tentativas >= 3 e acertos < 75%
```

Os títulos de nível são Calouro, Estagiário, Bacharel, Advogado, Procurador e Ministro (do sexto em diante permanece Ministro). As conquistas são avaliadas em `stats` e anunciadas pelo front (checkAch): primeiro, cartoes50, cartoes200, seq3, seq7, estrela, tres, mapa, nivel5 e aprovado.

6.3 Minutos estudados, ritmo e estimativa de conclusão

Minutos estudados (route_data): ordena os logs da trilha; entre duas respostas seguidas conta-se o intervalo se for de até 5 minutos; resposta isolada (primeira ou após pausa maior) vale 60 segundos. Os minutos por dia alimentam o gráfico dos últimos 7 dias. O resto é calculado no navegador por `analyze()`:

```
PACE = {1:40, 2:32, 3:24, 4:18, 5:12}          // cartões/hora na 1ª passada, por dificuldade
base(t)  = 60 / PACE[dificuldade]                // minutos por cartão
exp      = média ponderada de base() pelas tentativas;   obs = minutos_estudados / tentativas
w        = min(tentativas, 60) / 60
```

```
f        = 1 + w × (clamp(obs/exp, 0.6, 2) − 1)           // ritmo real
g        = acertos < 60% ? 1.25 : acertos >= 85% ? 0.9 : 1  // acertos
est(t)   = blocos×3 + visuais×4 + cartões × base(t) × 3 × f × g
rem(t)   = est(t) × (1 − dominados/total)
fim      = agora + (rem_total / meta_semanal_em_min) × 7 dias;   folga = prazo − fim
status   = concluída | sem_prazo | no_prazo (folga ≥ 0) | atenção (> −7 dias) | atrasado
ritmo %  = conteúdo% / prazo_decorrido%  (limitado a 150)
```

O cronograma semanal distribui **rem** de cada assunto pelas semanas, com capacidade igual às horas por semana do plano, até 52 semanas. Desativar a trilha congela o relógio (paused_at); reativar soma o tempo parado a start_ts e ao prazo (`resume_plan()`).

6.4 Jogo bônus: validação no servidor

```
$score = clamp(score, 0, 5000);  $kind in (snake, car, tetris)
$hoje   = partidas nas últimas 24 h;  $lastg = ts da última partida
$rows   = últimas N notas (N = game.streak)
$streakOk = count($rows) >= N  &&  nenhuma nota < 2  &&  nenhuma anterior à última partida
if ($hoje >= game.daily || !$streakOk)  → {xp: 0, note: ...}
$xp = min(game.max_xp, intdiv($score, 10));   // grava game_log e soma ao XP do usuário
```

6.5 Limites e proteções por contagem

Controle	Regra	Onde
Login	8 tentativas por IP em 10 min → 429; atraso de 0,3 s a cada erro	api.php (login), tabela attempts
Troca de senha	6 tentativas por usuário em 10 min; atraso de 0,4 s	api.php (pwd_change)
Trilhas por IA	ia_limit por usuário em 24 h (admin isento) → 429	route_create
Aulas simultâneas	3 em “enviando” ou “processando” por usuário → 429	media_init
Tamanho de aula	media_max_mb (padrão 500)	media_init
Jogos	game.daily em 24 h; XP ≤ game.max_xp	game_result
Pacote importado	JSON de até 3 MB	a_pkg_import
Provedores	No máximo 8	a_ia_save
Tokens por geração	1 a 50	a_token
PDF em material	Até 4 MB	material_save

7. Camada de IA

Todo o código está em `inc/ia.php`. A API pública da camada é `ai_complete($req, $accept, $role)`: recebe o pedido, uma função que valida e normaliza a resposta e o papel da etapa, e devolve o resultado já normalizado, o provedor que respondeu e se a busca foi usada.

7.1 Provedores e adaptadores

Tipo	Endpoint	Detalhes de implementação
`anthropic`	POST {base}/v1/messages (padrão https://api.anthropic.com)	Cabeçalhos x-api-key e anthropic-version 2023-06-01. PDFs viram blocos document (base64). Busca: ferramenta web_search_20250305 com max_uses 8. Se stop_reason for pause_turn, reenvia a conversa até 4 vezes para concluir. Modelo padrão: claude-sonnet-5-5.
`openai`	POST {base}/chat/completions (padrão https://api.openai.com/v1)	Authorization Bearer opcional (Ollama local não exige). Se o provedor recusar max_tokens (erro 400 citando o campo), repete com max_completion_tokens. Não lê PDF: avisa no prompt. Sem busca. Modelo padrão: gpt-4o-mini.
`gemini`	POST {base}/models/{modelo}:generateContent (padrão generativelanguage.googleapis.com/v1beta)	Cabeçalho x-goog-api-key. PDFs como inline_data. Busca: ferramenta google_search. Modelo padrão: gemini-2.5-flash.

`http_do()` é o cliente cURL (seguimento de até 4 redirecionamentos, timeout configurável, 900 s nas chamadas de IA, conexão de 15 s). Falhas HTTP viram `IaHttp($codigo, $mensagem)`. O campo `user` do pedido pode ser uma função `fn(bool $busca)`: o prompt muda conforme a busca esteja ou não disponível (search_rule).

7.2 Contingência e disjuntor

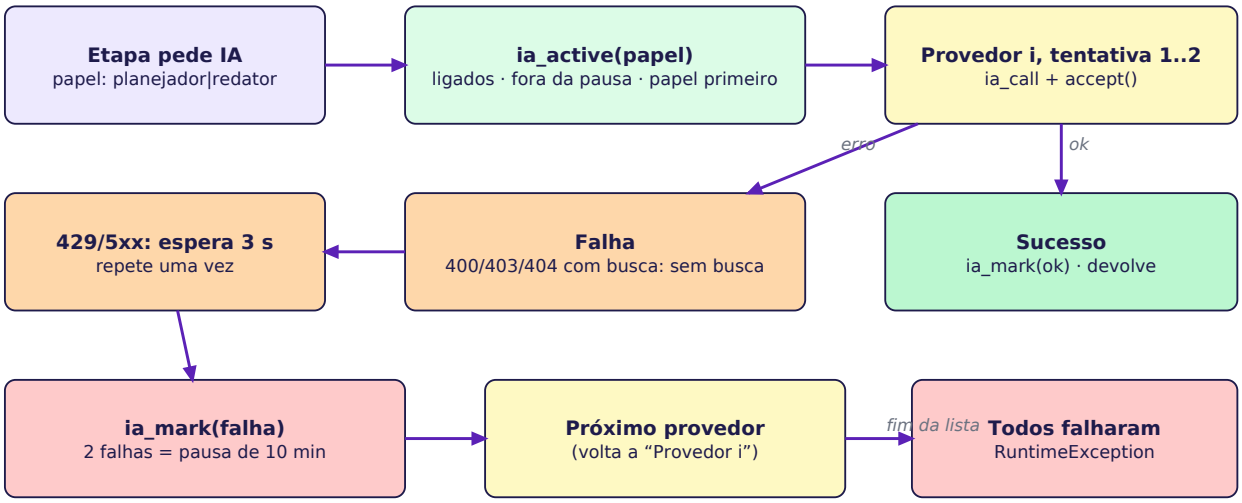


Figura 4 — Fluxo de ai_complete. A exceção final é tratada pelo orquestrador: modo agente ou esqueleto.

```
function ia_mark(string $id, bool $ok): void {
    $h = jd(setting('ia_health', ''));
    if ($ok) unset($h[$id]);
    else { $f = ((int)($h[$id]['fails'] ?? 0)) + 1; $h[$id] = ['fails' => $f, 'until' => $f >= 2 ? time() + 600 : 0]; }
    set_setting('ia_health', json_encode($h));
}
// ia_active(): filtra desligados e em pausa; se TODOS estiverem em pausa, usa a lista inteira (tenta mesmo assim);
// ordena por papel: igual ao da etapa (0), "qualquer" (1), outro (2).
```

> **Nota** Um erro de formato (a resposta não passa em accept()) é tratado como falha "não transitória": o provedor é marcado e o sistema passa ao próximo sem esperar. Erros de rede, 429 e 5xx esperam 3 s e repetem uma vez.

7.3 Extração e normalização de respostas

Modelos diferentes devolvem JSON com variações. `extract_json()` remove cercas de código, localiza o primeiro objeto equilibrando chaves e aspas, e tenta de novo sem vírgulas sobrando. Em seguida as funções `norm_*` aceitam sinônimos de campo (por meio de `first($arr, [chaves…])`), em português e inglês, e completam o que faltar com valores seguros:

Função	Aceita (exemplos de sinônimos)	Garante
`norm_route`	topics | assuntos | topicos | modules | unidades; cards | cartoes | flashcards; summary | resumo | conteudo	Rota válida: nome, emoji, estratégia, bibliografia e assuntos com cartões e visuais.
`norm_outline`	Idem + focus | foco | descricao | objetivo por assunto	Plano da etapa 1 com o foco de cada assunto.
`norm_topic`	summary | resumo | texto; mnemonic | mnemonico | dica; visuals | visuais	Pelo menos resumo ou cartões; limites: 8 blocos, 12 cartões, 3 visuais.
`norm_card`	Par [q, a] ou objeto {q|pergunta|question, a|resposta|answer}	Par de textos não vazios.
`norm_expand`	cards, visuals	Cartões novos e um resumo visual.
`norm_lesson / norm_bib_response / norm_deep`	Esquemas de aula, bibliografia e aprofundamento	Estruturas completas, com "a confirmar" quando não há verificação.
`norm_deepdive`	Aprofundamento de tópico (card ou tema)	Título, perguntas/respostas ou resumo e pontos.

A dificuldade pode vir como número (1–5) ou texto (fácil, médio, difícil) e é convertida; textos de resumo podem vir como lista ou parágrafo e são divididos em blocos.

7.4 Orquestrador de trilhas (run_route_job)

1. **Preparar**: `prep_videos()` (para links do YouTube, extrai o id com yt_id() e tenta obter as legendas; sem elas, avisa para colar a transcrição) e `prep_materials()` (junta o texto dos materiais selecionados e anexa PDFs até 4 MB como documentos). Calcula o orçamento de assuntos: horas = (prazo − hoje) em semanas × horas/semana (20 se não há prazo); *máx = clamp(round(horas / 2,5), 4, 10)* e *mín = máx − 2* (nunca abaixo de 3). Grava em jobs.payload o pedido e o prompt do modo agente, para permitir retomar.
2. **Planejar** (papel planejador, com busca): outline_prompt pede estratégia, bibliografia e a lista de assuntos com foco; norm_outline valida.
3. **Escrever** (papel redator, com busca): um assunto por vez com topic_prompt, que leva o plano, os títulos dos demais assuntos (para não repetir) e a bibliografia escolhida. Um assunto que falha após toda a contingência vira rascunho (skeleton_topic) e é anotado em progress.falhas; não derruba a trilha.
4. **Montar**: finish_route() grava a trilha com install_package (kind tema, material ou video), cria o plano (prazo, horas), marca como trilha atual do usuário e fecha o job (status pronto).

Estados de jobs.status: *gerando*, *manual* (IA indisponível: o front mostra o modo agente), *erro* e *pronto*. Estados de cada etapa: espera, andamento, ok, parcial, erro.

7.5 Modo agente e esqueleto

- **route_manual**: recebe o texto colado, aplica extract_json e norm_route e chama finish_route com a origem “agente externo”. O prompt entregue ao usuário é o route_prompt completo (trilha inteira de uma vez), com instruções para a IA externa marcar *verificada=true* só no que confirmar.
- **route_skeleton**: skeleton_route() cria um assunto por tema informado (ou assuntos genéricos), com cartões-modelo, marcados como rascunho em packages.sources.
- **topic_expand**: gera 4 a 6 cartões e 1 resumo visual para um assunto, sem repetir as perguntas existentes; sem IA devolve o prompt para o modo agente; topic_expand_apply aplica a resposta colada (apply_expand).

7.6 Prompts: contrato de saída

Todos os prompts pedem “somente JSON”, em português do Brasil, com textos curtos para baixa visão. O esquema da etapa 1 e o da etapa 2 são:

```
// Etapa 1 (planejamento)
{"name":"","emoji":"","goal":"","objective":"","tags":[""],
 "strategy":[{"tecnica":"","como":"","frequencia":""}],
 "biblio":[{"tipo":"livro|artigo|norma|site|video|material próprio","titulo":"","autores":"",
            "ano":2025,"url":"","por_que":"","verificada":true}],
 "topics":[{"title":"","emoji":"","difficulty":3,"law":"referências curtas","focus":"o que cobrir"}]}

// Etapa 2 (um assunto)
{"summary":[""],"mnemonic":"","refs":[""],"cards":[["pergunta","resposta"]],
 "visuals":[{"emoji":"","title":"","points":[""],"pitfall":"","example":""}]}
```

A regra de pesquisa (`search_rule`) muda conforme a busca esteja disponível: com busca, exige referências atuais; sem busca, manda marcar *verificada=false* no que não for certo. URLs só entram se conhecidas ou encontradas.

8. Aulas em áudio e vídeo

8.1 Upload em partes

1. **media_init**: valida a extensão (lista MEDIA_EXT: mp3, m4a, wav, ogg, oga, opus, flac, aac, webm, mp4, mov, mkv, avi, mpeg, mpg, 3gp, wma, wmv), o tamanho (media_max_mb) e o limite de 3 aulas simultâneas; cria o registro media com status *enviando* e devolve o id.
2. **media_chunk** (POST bruto, id e n na URL): o front corta o arquivo em partes de 4 MB e envia uma a uma, com até 3 tentativas por parte. Cada parte é gravada em media/<id>/ com nome numérico.
3. **media_done**: confere a contagem e o tamanho, concatena as partes no arquivo final (nome gerado pelo servidor, nunca o do usuário), apaga as partes e inicia `run_media_job()` em segundo plano.

8.2 Pipeline (run_media_job)

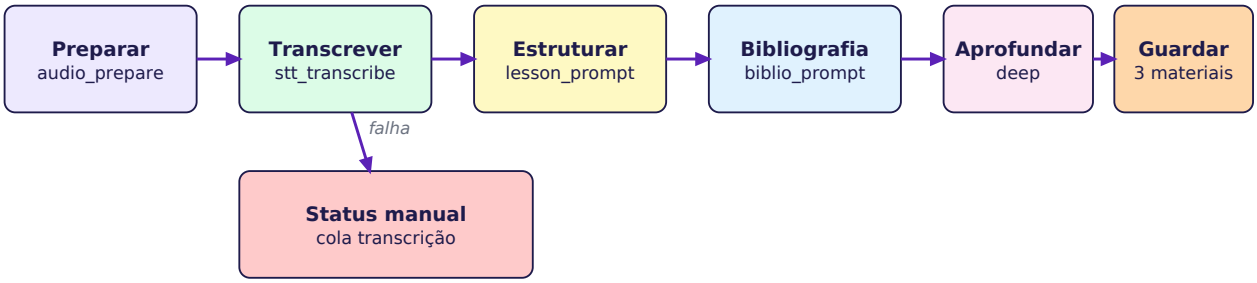


Figura 5 — Etapas da aula. Cada etapa grava seu estado em media.progress; o front consulta media_job.

Etapa	O que faz	Persistência
preparar	Com ffmpeg: extrai áudio mono, 16 kHz, mp3 (arquivo muito menor). Sem ffmpeg: usa o original.	media.progress
transcrever	stt_transcribe percorre os agentes com transcrição (ia_stt_active) na ordem de prioridade. Falha total → status *manual*.	media.transcript, progress.agentes
estruturar	Divide a transcrição em blocos de até ~80 mil caracteres; cada bloco gera capítulos, conceitos e fontes citadas; lesson_merge junta; lesson_fallback se a IA falhar.	media.structure
bibliografia	Com busca na web: detalha cada obra ou lei citada (autor, edição, ano, url, verificada). Sem busca: marca como a confirmar.	media.biblio
aprofundar	lesson_deep_prompt: estado da arte do tema, ligado à bibliografia.	media.deep
biblioteca	Cria três materiais (kind aula): transcrição, resumo com capítulos e bibliografia, aprofundamento.	materials, media.material_ids

Estados de media.status: *enviando*, *processando*, *manual*, *pronto* e *erro*. O arquivo original é removido ao concluir; ficam a transcrição e os resultados no banco. Se a IA estiver desligada, o pipeline salva a transcrição e um resumo básico sem chamar agentes.

8.3 Transcrição (STT)

Agente	Algoritmo	Limites e continuação
Gemini (stt_gemini)	Arquivos pequenos: inline_data no generateContent. Grandes: API de arquivos (gemini_upload, upload resumível) e depois generateContent com a referência.	Aulas longas são transcritas por partes com instrução de continuar do ponto anterior; marcações de tempo fmt_ts.
OpenAI (stt_openai)	POST multipart em /audio/transcriptions com o modelo de transcrição.	Máximo de 25 MB por chamada: acima disso, divide com ffmpeg em segmentos e junta os textos; sem ffmpeg, lança exceção e o agente é pulado.

Se nenhum agente servir, a aula entra em *manual*. A rota `media_text` aceita a transcrição colada ou o conteúdo de .srt/.vtt (limpo de numeração e marcações) e retoma a partir da etapa “estruturar”.

9. Mapa de conhecimento e aprofundamento

9.1 Dados do grafo (graph_data)

`graph_data($uid, $pid)` devolve, em uma única consulta composta, tudo o que o front precisa; o nível de complexidade é aplicado no cliente.

Campo	Conteúdo
`route`	id, nome, emoji, objetivo da trilha.
`topics`	id, título, emoji, dificuldade, refs, n (cartões), k (dominados), due (a revisar), tries, ok; calculados: stars, acc (acertos %), weak (tentativas ≥ 3 e acertos < 75%), review (weak ou há cartões vencidos com tentativas).
`cards`	Até 600 cartões: id, topic_id, q, a, due (vencido ou sem progresso).
`biblio`	Bibliografia da trilha (itens do packages.biblio).
`deep`	Aprofundamentos salvos do usuário nessa trilha.
`related`	Até 8 outras trilhas visíveis ao usuário (can_view), entre as 120 mais recentes, com pontuação.

```
score(outra) = 2 × |etiquetas em comum|          // tags, comparadas em minúsculas
             + 3 × |obras em comum|              // títulos da biblioteca, minúsculas
             + 5 × (origem comum ou cópia)       // origin_id igual, ou uma é cópia da outra
// só entram score > 0, ordenadas por score decrescente
```

9.2 Renderização (front-end)

- **Graph3D(canvas, nodes, edges, opções)**: Canvas 2D sem bibliotecas. Posição inicial dos nós em espiral de Fibonacci sobre uma esfera de raio 170, com ruído. Física de molas: repulsão entre todos os pares, molas nas arestas (comprimento por tipo de ligação), amortecimento e centralização. Projeção em perspectiva com distância focal F = 520; rotação por ângulos ay e ax; zoom por roda, pinça e teclas. Seleção por teste de acerto no ponto projetado (hit test), rótulos com tamanho em função da profundidade.
- **Desempenho**: respeita prefers-reduced-motion (sem giro automático), limita o número de nós pela configuração (max_nodes, padrão 160) e encerra o laço de animação quando o canvas é removido da página.
- **Níveis**: 1 trilha e tópicos; 2 acrescenta cartões (6 por tópico, priorizando os a revisar); 3 acrescenta bibliografia (ligada pelos refs) e aprofundamentos; 4 acrescenta trilhas relacionadas. Os tópicos incluídos podem ser filtrados por chips; o filtro “só para revisar” usa review.
- **Fluxograma**: lista vertical (trilha → tópicos → cartões e aprofundamentos), sem canvas; a mesma lista acessível existe sob o desenho 3D para leitores de tela.
- **Cores**: nós de tópico recebem a cor de SC[stars]; borda tracejada indica reforço; o avatar anima sobre o tópico atual.

9.3 Aprofundamento por tópico

`deep_generate` recebe o tópico (e opcionalmente o cartão), o tipo (*card* ou *tema*) e um foco. `deep_prompt()` monta o pedido com o resumo do tópico, a bibliografia principal da trilha (priorizando as obras listadas em topics.refs), o objetivo e o foco; a IA responde em JSON normalizado por `norm_deepdive()`. `deep_save()` grava:

- em **deepdives** (sempre), com topic_id, card_id, kind, título, q, a, resumo, pontos e refs;

- em **materials** (só os temas), kind texto, com a lista de fontes, para entrar na biblioteca e servir de base a novas trilhas;
- nas **fontes marcadas como a confirmar** quando a busca não estava disponível.

Sem IA, deep_generate devolve o prompt pronto para o modo agente e `deep_apply` recebe o JSON colado e chama o mesmo deep_save.

10. Front-end

10.1 Organização do app.js

Bloco	Conteúdo
Utilitários	$ (querySelector), el(tag, atributos, …filhos) que cria elementos com textContent, sv() para SVG, api() e apiRaw() (fetch com X-CSRF), notify(), fmt*.
Estado S	Usuário, CSRF, nome, preferências, edições, configuração do jogo e do mapa, indicador de offline.
Preferências	applyPrefs() grava variáveis CSS (--fs, --lh, --ls), data-theme e salva em localStorage e no servidor (setPref com atraso).
Acessibilidade e voz	say() (speechSynthesis), barra de ajustes, “Ler ao focar”, toggleTools() e o desenho padrão.
Gamificação	lvInfo(), gain(), toast(), burst(), hud(), checkAch() e a tabela ACH.
Fila offline	queueGrade() e flush(): notas pendentes em localStorage (ecp), enviadas por grades quando há rede; a fila de cartões da última sessão fica em cache para reabrir sem conexão.
Roteador	render() lê o hash, escolhe a view e a carrega; trata 401 (volta ao login) e erros de rede.
Views	vChooser, vHome, vDash, vRoute, vRoutes, vNewRoute, vImport, vTopics, vTopic, vStudy, vLibrary, vDeck, vAch, vExplore, vPublic, vGroups, vGroup, vMaterials, vLessons, vLesson, vGraph, vDeepLib, vMore, vSettings, vAccount, vSources e vAdmin.
Gráficos	Mapa da trilha (SVG), Graph3D e fluxograma, gráfico de barras e medidores do painel.
Jogos	Motor comum em Canvas (grade de pixels, paletas, laço com passo fixo, toque e teclado) e três jogos: snake, car e tetris.
Início	Registro do service worker, boot() e primeira renderização.

10.2 Roteador: rotas e telas

Hash	Tela	Observação
`(vazio) / qualquer outro`	vDash	Início; sem sessão: vHome (login, cadastro, pedido de acesso).
`#/versao`	vChooser	Escolha da edição; também aparece quando não há edição definida.
`#/topicos, #/topico/:id`	vTopics, vTopic	Conteúdo e um assunto.
`#/resumos, #/resumo/:id`	vLibrary, vDeck	Resumos visuais.
`#/revisar/:modo/:id`	vStudy	modo due, topic ou exam.
`#/trilhas, #/trilhas/nova, #/trilhas/importar`	vRoutes, vNewRoute, vImport	Aceita ?tema= e materiais pré-selecionados (trilha da aula).
`#/trilha/:id`	vRoute	Painel da trilha.
`#/explorar, #/comunidade/:id`	vExplore, vPublic	Comunidade.
`#/grupos, #/grupo/:id`	vGroups, vGroup	Grupos de estudo.
`#/materiais`	vMaterials	Biblioteca.
`#/aulas, #/aula/:id`	vLessons, vLesson	Aulas.
`#/grafo/:id?nivel=&modo;=`	vGraph	Mapa de conhecimento.
`#/aprofundamento/:id`	vDeepLib	Biblioteca de aprofundamento.
`#/conquistas, #/ajustes, #/mais, #/fontes`	vAch, vSettings + vAccount, vMore, vSources	Conquistas, ajustes e senha, menu “Mais”, fontes.
`#/admin/:aba`	vAdmin	Abas: users, tokens, requests, packages, versoes, jogo, ia, dels, content.

10.3 Edições, temas e desenho padrão

- A edição é uma classe do body: `portal` ou `acc` (acessível), definida pelo servidor (shell.php) a partir do cookie ed, do parâmetro ?trocar e da configuração editions; o localStorage ec_ed repete a escolha para o primeiro desenho.
- `assets/theme.js` roda antes do app.js: lê ec (preferências em cache) e `ec_tools` e aplica data-theme, tamanho de letra e as classes de html e body, para não haver “piscada”.
- Temas (data-theme): **claro** (rótulo Sol), **noite** e **azul** (rótulo Mar). Variáveis CSS principais: --bg, --fg, --card, --mut, --p, --ac, --acf, --ok, --bad, --sh, --t1 a --t6 (cores de tópico/tipo), --fs, --lh e --ls.
- **Desenho padrão**: classe `std` no html (acessibilidade desligada, ec_tools = 0): carrega std.css por último, reduz fonte para 15 px, paleta neutra, menu lateral compacto ou abas no celular e remove animações. A classe `tools-off` do body oculta a barra de ajustes.
- nonav: o body recebe esta classe enquanto o menu está vazio (tela de escolha e login); #nav fica display:none e a coluna do menu não é reservada.
- `immersive`: durante a revisão, esconde o menu e dá toda a largura à grade pergunta/resposta; gaming bloqueia os atalhos de revisão enquanto o jogo roda.

10.4 Telas principais

- **vStudy (revisão)**: sessão de cartões com estado local (i, shown, hit, streak). Pergunta à esquerda e resposta à direita em ≥ ~36 rem, empilhadas abaixo disso; a resposta fica em placeholder opaco até showAns(). Teclas Espaço e 1–4; botão do tópico abre o painel de aprofundamento. Ao completar a sequência definida em S.game.streak com notas ≥ 2, abre o jogo. Ao final: resumo e "recordação livre".
- **Mapa da trilha**: SVG montado a partir das estrelas; o caminho é uma curva por pontos P; o trecho percorrido recebe a animação draw; cada fase é um nó com rótulo ARIA completo ("Fase 3: título. 2 de 3 estrelas. Liberada."); há lista ordenada equivalente.
- **vNewRoute**: formulário, seleção de materiais e vídeos, limite diário exibido, acompanhamento por consulta periódica de route_job e exibição de modo agente (textarea com o prompt e campo para colar o JSON).
- **vLessons e vLesson**: envio em partes com barra de progresso, lista de aulas, etapas animadas, modo manual e o botão "Criar trilha com esta aula".
- **vAdmin**: abas com ações em lote por meio do helper `act(rota, corpo, depois, rótulo, classe, confirmação)`, que cria um botão já ligado à API.

10.5 PWA e cache

`sw.js` (versão de cache **ec-v4**) pré-carrega a casca (app.css, portal.css, std.css, mark.svg, app.js, theme.js, icon.svg, manifest) e usa: **rede primeiro com cópia em cache para navegação** (se a rede falha, serve a página guardada ou a raiz); **cache primeiro com atualização em segundo plano** para os demais GET do mesmo domínio. Nunca intercepta POST, api.php nem install.php. Para forçar a atualização do aplicativo instalado, aumente a constante V. A estratégia de "cache primeiro" significa que uma nova versão do app.js só aparece na abertura seguinte à do primeiro carregamento.

Chave localStorage	Uso
`ec`	Cópia das preferências (lida pelo theme.js antes do app).
`ec_ed`	Edição escolhida (portal ou acessivel).
`ec_tools`	1 ou 0: barra de acessibilidade visível; se ausente, segue prefs.a11y.
`ecu`	Último usuário (permite abrir offline com o último estado).
`ecp`	Fila de notas pendentes de envio (offline).
`ecr`	1 ou 0: "Ler ao focar" ligado.
`(chave por fila)`	Cartões da última fila de revisão, para uso sem conexão.

10.6 Segurança no navegador

- Todo texto vindo do servidor entra por textContent ou createTextNode (função el()), nunca por innerHTML.
- CSP sem script ou estilo inline: o dinâmico é feito por classList, setAttribute de atributos seguros e CSSOM (style.setProperty).
- O token CSRF vem do boot e segue em X-CSRF em toda chamada POST; um 419 faz o front orientar o recarregamento.
- Links externos de materiais usam rel=noopener e abrem em nova aba.

11. Instalador

`install.php` é uma página PHP única, com `assets/install.js` e `install.css`. Enquanto não existe config.php ela atende; depois que existe, redireciona ao app.

1. **Verificações**: PHP_VERSION_ID ≥ 80100, extensão pdo_sqlite, cURL e ffmpeg (opcional). Define a pasta de dados por `data_dir()`: *<pasta>_dados* ao lado da pasta pública quando gravável; senão *ROOT/data*.
2. **Chave de instalação**: se existir `install.key`, o formulário exige o conteúdo do arquivo.
3. **Teste de agentes**: `install.php?do=test` (POST JSON) executa uma chamada curta ao agente informado, ou ao pré-configurado (preset_key), e devolve ok ou o erro do provedor; o assistente usa para mostrar o estado antes de instalar e para animar a cadeia de socorro.
4. **Instalação**: transação única: cria o banco (schema), o administrador (password_hash), grava settings (editions, agentes, media_max_mb), instala os pacotes da pasta pacotes/ quando marcado (seed). Só então grava config.php com `fopen(…, 'x')`: o modo x falha se o arquivo já existir, impedindo instalação dupla e disputa entre duas abas.
5. **Limpeza**: apaga install.key e inc/preconfig.php (as chaves já estão no banco). Em caso de erro, desfaz a transação e remove o banco recém-criado se não houver config.php.

`inc/preconfig.php` retorna um array com a chave *agents* (id, type, label, key, model, base_url, role, search, stt, enabled). O instalador mostra os agentes sem expor as chaves (preset_key resolve a chave no servidor pelo id).

12. Segurança técnica

Ameaça	Mitigação	Observações / limites
Injeção de SQL	Todas as consultas usam PDO com parâmetros (função q()). Fragmentos dinâmicos são números convertidos com (int) ou listas fechadas.	Ao escrever SQL novo, nunca concatene entrada do usuário.
XSS	textContent no front; CSP self para script e estilo; nosniff; frame-ancestors none.	Evite innerHTML. Textos de IA e de usuários são sempre tratados como texto.
CSRF	Token por sessão no cabeçalho X-CSRF, comparado com hash_equals; SameSite Lax.	GET nunca altera estado (exceto efeitos colaterais de leitura como estatísticas).
Sessão	Cookie httponly, SameSite Lax, Secure em HTTPS; session_regenerate_id ao entrar e ao trocar a senha; nome ec; path da aplicação.	Cabeçalho X-Forwarded-Proto é aceito para detectar HTTPS atrás de proxy: configure o proxy para sobrescrevê-lo.
Força bruta	Janela de 10 min na tabela attempts; atrasos por erro.	A chave do login é o IP; atrás de proxy sem repasse de IP todos compartilham o limite.
Senhas	password_hash (PASSWORD_DEFAULT) e password_verify; mínimo de 8 caracteres.	Sem verificação de senhas vazadas nem autenticação em dois fatores.
Autorização	user() e admin() no topo de cada rota; verificações de posse (owner_id, can_view, visible_pkg) antes de ler ou alterar objetos.	IDOR é o risco mais provável em rotas novas: sempre confira o dono.
Upload	Extensão em lista fechada, limite de tamanho, nome do arquivo gerado no servidor, pasta fora do site, conteúdo nunca executado.	O tipo MIME declarado pelo cliente é só informativo (mime_of mapeia pela extensão).
SSRF	As únicas URLs buscadas pelo servidor são as dos provedores (definidas pelo administrador) e as legendas de vídeos do YouTube (id extraído por yt_id).	Se permitir URLs arbitrárias no futuro, bloqueie IPs privados e redirecionamentos.
Vazamento de chaves	Chaves só no servidor; a API devolve os 4 últimos caracteres; .htaccess nega config.php e SQLite; preconfig.php apagado.	O banco contém as chaves em texto: proteja a pasta e os backups.
Abuso de IA	Limite diário por usuário, limite de aulas simultâneas e de tamanho, teto de tokens por chamada.	Sem medição de custo por usuário.
Exposição de erros	Mensagens de erro de IA são encurtadas; exceções internas viram mensagens genéricas na maioria das rotas.	Em produção deixe display_errors desligado.

13. Formatos de dados

13.1 Pacote de conteúdo (importação pelo administrador)

```
{
  "slug": "direito-administrativo",      // minúsculas, números e hífen; único
  "name": "Direito Administrativo", "goal": "…", "emoji": "⚖", "objective": "…",
  "tags": ["concursos", "direito"],       // até 8
  "strategy": [{"tecnica": "", "como": "", "frequencia": ""}],
  "biblio":   [{"tipo": "livro", "titulo": "", "autores": "", "ano": 2024, "url": "", "por_que": "", "verificada": true}],
  "sources":  {},
  "topics": [{
      "title": "Atos administrativos", "emoji": "", "difficulty": 3, "law": "Lei 9.784/99",
      "summary": ["bloco 1", "bloco 2"], "mnemonic": "…", "refs": ["título de obra da biblio"],
      "cards":   [["pergunta", "resposta"]],
      "visuals": [{"emoji": "", "title": "", "points": ["…"], "pitfall": "", "example": ""}]
  }]
}
```

Regras de install_package: exige slug, name e ao menos um assunto com título; o slug duplicado é recusado para pacotes do sistema e recebe sufixo aleatório para trilhas de usuário; difficulty é limitada a 1–5; cartões precisam ser pares com 2+ itens; tudo é gravado numa transação (falhou, desfaz). Aceita também *bibliography* no lugar de *biblio*.

13.2 Trilha exportada (.trilha.json)

```
{
  "format": "estudo-claro-trilha", "version": 1, "exported_at": "2026-10-17T12:00:00-03:00",
  "route": { …mesmo formato do pacote, sem slug… },
  "plan": {"deadline": "2026-12-01", "hpw": 5, "elapsed_s": 86400, "remaining_s": 3000000, "status": "ativa"},
  "progress": [ [ [ease, ivl, reps, lapses, due] | null, … por cartão ], … por assunto ],
  "integrity": "sha256 de json_encode(route)"
}
```

Na importação, o hash é recalculado; se divergir, a trilha é importada mesmo assim com um aviso. O progresso é restaurado por posição (assunto i, cartão j), e o plano é reconstruído com o tempo restante (remaining_s) em vez da data, para que o prazo relativo seja mantido.

13.3 Resposta de uma aula (media)

```
structure: {"titulo":"", "resumo":["…"], "capitulos":[{"titulo":"","inicio":"00:00","pontos":["…"]}],
            "conceitos":[{"termo":"","definicao":""}], "fontes_citadas":["…"]}
biblio:    [{"tipo","titulo","autores","ano","edicao","url","verificada","observacao"}]
deep:      {"titulo":"", "resumo":["…"], "pontos":["…"], "questoes":["…"]}
```

> **Nota** Os campos acima são os normalizados por norm_lesson, norm_bib_response e norm_deep; as funções aceitam sinônimos, então a IA não precisa seguir exatamente os nomes.

14. Testes e garantia de qualidade

14.1 Estratégia

O front-end é testado de ponta a ponta em navegador real (Playwright, Chromium) contra uma **prévia funcional**: um único HTML que contém o app.js e os CSS reais mais um servidor simulado em memória (mock) no lugar do api.php. Assim, o mesmo código de interface roda nos testes e na demonstração. O servidor PHP em si ainda não foi executado em ambiente com PHP neste projeto.

Bateria	Testes	Cobertura
e2e.py	59	Escolha de versão, login, trilhas, revisão, notas, mapa da trilha, gamificação, painel, explorar, comunidade, grupos, materiais, exportar e importar, administração.
e2e2.py	25	Troca de senha (validações e limites), desenho padrão (liga/desliga, persistência, menu e abas).
e2e3.py	23	Aulas: envio em partes, etapas, transcrição manual, resultado, trilha a partir da aula.
e2e4.py	37	Menu vazio, revisão lado a lado (larguras), aprofundamento, grafo 3D e fluxograma (níveis, filtros), jogos, administração do jogo e do mapa.
Total	144	Zero falhas e zero erros de console nas quatro baterias, na última execução.

14.2 Como a prévia é montada

```
python3 /tmp/build_preview.py    # junta app.css, portal.css, std.css, app.js e o mock em estudo-claro-previa.html
python3 /tmp/build_wrapper.py    # atualiza a moldura de celular (estudo-claro-app-visual.html)
python3 /tmp/e2e.py              # (e e2e2, e2e3, e2e4) — Playwright; imprime TOTAL, FALHAS e ERROS
```

O mock substitui a função api() do app.js: mantém usuários, trilhas, progresso, aulas, grupos e configurações em memória (com localStorage), reproduz as regras principais (SM-2, estrelas, XP, limites) e simula a IA com respostas fixas; a caixa “Simular IA fora do ar” força o modo agente. Ele serve para validar a interface, não para provar o PHP.

14.3 Verificações estáticas feitas no PHP

- Balanceamento de chaves e parênteses em todos os arquivos PHP.
- Varredura de funções duplicadas entre arquivos incluídos juntos: encontrou `deep_prompt` definida em aulas.php e em mapa.php (erro fatal “Cannot redeclare”); a de aulas foi renomeada para `lesson_deep_prompt`.
- Varredura de chamadas a funções inexistentes (nenhuma encontrada além de funções nativas).
- Requisito de versão: o tipo de retorno `never` exige PHP 8.1; o instalador e a documentação foram ajustados de 8.0 para 8.1.

14.4 O que falta testar (recomendado antes da produção)

Lacuna	Como cobrir
Execução real do PHP	Subir com `php -S localhost:8080`, rodar `php -l` em todos os arquivos, instalar pelo assistente e repetir as baterias Playwright apontando para o servidor real em vez da prévia.
Regras de negócio no servidor	PHPUnit (ou scripts simples) para review() (tabela do SM-2), extract_json e norm_* (respostas com cercas, vírgulas sobrando e sinônimos), install_package, export/import_route (round-trip), validação do jogo (game_result).
Provedores de IA reais	Teste de contrato com chaves de teste: uma chamada por provedor e por papel, com e sem busca; simular 429, 500 e resposta malformada para validar o disjuntor.
Transcrição real	Aulas curtas, médias e longas com Gemini e Whisper; arquivo acima de 25 MB com e sem ffmpeg.
Concorrência	Várias gravações simultâneas no SQLite (busy_timeout e WAL) e dois navegadores no mesmo usuário.
Segurança	Teste de IDOR por rota (usuário A acessando recurso de B), CSRF sem cabeçalho, limites de tentativa, upload malicioso.
Acessibilidade	Revisão com leitores de tela (NVDA, VoiceOver, TalkBack) e auditoria automatizada de contraste e foco.

```
# Roteiro mínimo de fumaça contra um servidor real
php -l api.php && for f in inc/*.php; do php -l "$f"; done
curl -s 'http://localhost:8080/api.php?r=boot' | head            # {"user":null,"csrf":"…"}
curl -s -c cj -b cj 'http://localhost:8080/api.php?r=boot' >/dev/null
CSRF=$(curl -s -c cj -b cj 'http://localhost:8080/api.php?r=boot' | python3 -c 'import sys,json;print(json.load(sys.stdin)
curl -s -c cj -b cj -H "X-CSRF: $CSRF" -H 'Content-Type: application/json' \
     -d '{"email":"admin@x.com","pass":"senha-do-admin"}' 'http://localhost:8080/api.php?r=login'
```

15. Como estender o sistema

15.1 Nova rota da API

1. Em api.php, acrescente `case 'minha_rota':` no switch. Comece com `$u = user();` (ou admin()).
2. Valide a entrada com str($in, 'campo', max), conversões (int) e listas fechadas (in_array estrito).
3. Se mexe em objeto de alguém, confira posse ou visibilidade (owner_id, can_view, visible_pkg).
4. Use q() com parâmetros. Em operações com várias gravações, envolva em transação (db()->beginTransaction()).
5. Termine com `out([...])`.
6. No front, chame `api('minha_rota', {body})` (POST) ou `{qs}` (GET). Documente a rota na tabela do capítulo 5 (a coluna Entrada é gerada, a descrição é manual).

15.2 Nova tela

1. Crie `async function vMinhaTela(...)` que devolve um nó (use el() e textContent).
2. Registre a rota no `render()` com um else if e o título.
3. Se for item de menu, acrescente em buildNav() (Portal e acessível têm listas próprias).
4. Cuide do foco (foque o h1 ao entrar) e de rótulos ARIA; confira nas três paletas e no desenho padrão.
5. Acrescente o novo arquivo estático à lista do sw.js e aumente a versão V, se for um arquivo novo.

15.3 Novo provedor de IA

1. Se for compatível com OpenAI (a maioria), use o tipo openai com o endereço base: não precisa de código.
2. Para um protocolo novo: acrescente a chave em `IA_TYPES`, um ramo em `ia_call()` que devolva {text, search}, um modelo padrão e, se tiver transcrição, um adaptador em aulas.php chamado por stt_transcribe.
3. Lance IaHttp(código, mensagem) para erros HTTP, a fim de o disjuntor e a regra “tentar sem busca” funcionarem.

15.4 Novo jogo bônus

1. Adicione o identificador à lista aceita em `clean_game()` e em `game_result` (lista fechada de jogos).
2. No app.js, implemente a função do jogo seguindo o contrato do motor: recebe o canvas e as opções (paleta, velocidade, duração) e devolve a pontuação por callback ao terminar.
3. Inclua o nome no seletor da aba Jogo e mapa e nos testes (e2e4.py).

15.5 Nova conquista, tema ou nível

- **Conquista**: avalie a condição em `stats` (api.php) devolvendo o código, e acrescente o código à tabela ACH do app.js com ícone, título e descrição.
- **Tema**: copie um bloco [data-theme=...] em app.css, defina as variáveis e acrescente o valor em clean_prefs() e no seletor de Ajustes.

- **Níveis**: ajuste a fórmula em lvInfo() (app.js) e em stats (api.php) nos dois lugares, e a lista LV de títulos.

15.6 Nova edição (versão da interface)

Uma edição é uma classe do body mais uma folha de estilo. Para criar uma terceira: acrescente o valor em *editions* (a_cfg_set e shell.php), a classe e o CSS, o item na tela de escolha (vChooser) e a lógica de ED().

16. Operação, desempenho e evolução

16.1 Configuração de produção

- PHP-FPM com opcache ligado, `display_errors=Off` e `log_errors=On`.
- Tempos: max_execution_time ≥ 120 e timeouts do proxy/FPM acima de 300 s (as chamadas de IA têm timeout de 900 s dentro de trabalhos em segundo plano).
- Pasta de dados em disco rápido, fora do site, com permissão 0750 para o usuário do PHP.
- Backup consistente do SQLite (comando .backup ou os três arquivos juntos) e retenção definida.

16.2 Desempenho

- As consultas mais pesadas (route_data, graph_data, queue) usam subconsultas por assunto; os índices i_prog (user_id, due) e i_logs (user_id, ts) cobrem o essencial.
- O grafo limita cartões a 600 e nós a max_nodes; o laço do canvas termina quando ele sai da página.
- Respostas de IA são o gargalo: o orquestrador já trabalha em segundo plano; não chame ai_complete dentro de rotas síncronas.
- Cache de estáticos: no-cache com revalidação (.htaccess) e service worker; considere ETag e compressão (gzip/brotli) no servidor.

16.3 Escalabilidade e migração de banco

O SQLite com WAL atende bem uma instituição pequena ou média, pois a leitura é concorrente e as gravações são curtas. Os sinais de limite são mensagens “database is locked” frequentes e trabalhos de IA simultâneos disputando gravações. Para migrar para PostgreSQL ou MySQL: (1) troque a criação do PDO em db() e remova os PRAGMAs; (2) adapte *INSERT OR IGNORE* e *ON CONFLICT … DO UPDATE* (a sintaxe de UPSERT muda no MySQL); (3) substitua *PRAGMA table_info* na migração por consultas ao information_schema; (4) revise *RANDOM()* (RAND() no MySQL), *char(10)* e *LENGTH/REPLACE* usados para contar blocos; (5) mantenha q() como camada única, o que concentra a mudança.

16.4 Observabilidade

O sistema não grava logs de aplicação próprios. Recomenda-se: registrar com error_log() as falhas de provedor (já há ia_health no banco), medir o tempo das rotas de IA e acrescentar contadores de uso (trilhas, aulas, minutos de transcrição) se for preciso controlar custo.

16.5 Limitações conhecidas

Limitação	Impacto	Caminho
PHP ainda não executado em teste automatizado	Podem existir erros de execução ainda não vistos (a verificação foi estática e pelo front com servidor simulado).	Executar o roteiro do capítulo 14.4 antes de abrir para usuários.
Segundo plano sem fila	Se a hospedagem encerra o processo, o trabalho para; o usuário retoma manualmente.	Adicionar um comando de linha (cron) que retome jobs parados.
Sem recuperação de senha por e-mail	Depende do administrador.	Integrar envio de e-mail e tokens de redefinição.
Sem dois fatores	Conta protegida só por senha.	TOTP para administradores.
Estimativa de conclusão no navegador	Cada cliente calcula; sem histórico.	Mover analyze() para o servidor se for preciso relatórios.
Um arquivo grande de JS e de CSS	Manutenção mais lenta.	Dividir em módulos ES quando for conveniente (continua sem build).
Conteúdo gerado por IA	Pode conter erros ou fontes não confirmadas.	Marcadores “a confirmar”, revisão humana e ferramentas de edição (aba Conteúdo).
Segredos no banco	Quem acessa o arquivo vê as chaves.	Criptografar a coluna com uma chave de ambiente (sodium_crypto_secretbox).

Apêndice A — Glossário técnico

Termo	Significado
Trilha / pacote	Registro em packages: um conjunto de assuntos (topics) com cartões e visuais. “Pacote” é o formato de importação; “trilha” é o que o usuário vê.
Edição	Variante de interface (portal ou acc) escolhida na tela de escolha.
Desenho padrão	Aparência compacta com acessibilidade desligada (classe std).
Dominado	Cartão com reps ≥ 2.
Estrelas	Níveis de domínio de um assunto: 33%, 66% e 100% dos cartões dominados.
SM-2	Algoritmo de repetição espaçada do SuperMemo, aqui adaptado com as quatro notas.
Planejador / redator	Papéis de agentes de IA nas etapas 1 e 2 do orquestrador.
Disjuntor	Mecanismo que tira um provedor do rodízio por 10 minutos após 2 falhas.
Modo agente	Fluxo manual: copiar o prompt, usar outra IA, colar o JSON.
STT	Reconhecimento de fala (speech-to-text): transcrição de aulas.
WAL	Write-Ahead Logging do SQLite: leitura concorrente com escrita.
CSP	Content Security Policy: política de carregamento de recursos do navegador.
CSRF	Falsificação de requisição entre sites; barrada pelo cabeçalho X-CSRF.
PWA	Aplicativo web progressivo: instalável e com cache offline.

Apêndice B — Lista de verificação para revisão de código

- A rota chama user() ou admin() antes de ler ou gravar? Confere posse e visibilidade?
- Toda consulta usa parâmetros? Nenhum campo do usuário é concatenado em SQL ou em comandos de shell (ffmpeg usa escapeshellarg)?
- Entradas têm limite de tamanho e valores permitidos em lista fechada?
- O front usa textContent e não innerHTML? Há estilo ou script inline (a CSP bloqueia)?
- Nova coluna entrou em $add (migração) e não só em CREATE TABLE?
- Chamadas de IA estão em segundo plano ou numa rota com tratamento de falha e modo agente?
- Novas telas têm foco, rótulos ARIA, lista equivalente para gráficos e funcionam nas três paletas e no desenho padrão?
- Nomes de funções PHP são únicos entre todos os arquivos incluídos juntos (lembre do caso deep_prompt)?
- O arquivo estático novo está no cache do service worker e a versão V foi aumentada?
- Os testes e2e foram atualizados e passam sem erros de console?